

RECORDING FLOW · BEFORE & AFTER

Making the recording feel instant.

What was slow, what we changed, and how we know it worked.

THE STARTING POINT

Tap **Done** and the screen would sit there for two or three seconds before moving on.

The freeze wasn't the network. The phone was doing too many things in a row before letting the UI move.

UNDER THE HOOD, BEFORE

Every step ran four heavy things back to back, on the main thread.

Stopping the audio finalized the m4a file. Then the motion-sensor buffer flushed to Firestore. Then the local recording list was rewritten on disk. Then the upload was queued. All serial. All blocking the screen.

On top of that, every tap on Record re-checked the microphone permission and probed the sensors, and the waveform held a three-second warm-up before it even appeared.

We let the UI move the moment you tap **Done**, and pushed everything heavy into the background.

The screen advances to the next prompt in a single frame. Finalizing the file, flushing the sensors, saving locally, and queueing the upload now run as a quiet pipeline behind it.

Permissions and the audio session are pre-warmed when the screen first mounts, so tapping Record costs nothing. The waveform shows from the first second.

Local saves write to an in-memory cache and persist to disk after the fact, so they're effectively free.

END-OF-SESSION UPLOAD

The final upload used to push recordings one after another. Now three go up at once.

Same storage contract, same retry behavior. We just stopped waiting on each file before starting the next one.

A SMALL ORGANIZATIONAL FIX

Each five-step run now carries one **session id**.

The timeline groups the recordings as "Session 1", "Session 2" instead of ten loose entries scattered across the day. Older recordings without an id are grouped by time and step number so nothing looks orphaned.

HOW WE MEASURED

Stopwatch on a real device, then a five-recording end-to-end run on Wi-Fi.

Per-step latency was measured from the tap on Done to the next prompt becoming visible, averaged across ten runs. Final upload time was measured end-to-end across five recordings on the same network. Typecheck and Expo Doctor were green across the change, so no SDK or library swap is hiding the numbers.

RESULTS

From a noticeable wait
to a single frame.

PER-STEP TRANSITION

~~2.4 s~~ → instant

FINAL UPLOAD (5 FILES, WI-FI)

~~14 s~~ → ~5 s

CODE PATH

unchanged

No SDK change. No library swap. Same recording semantics, same storage layout.

TAKEAWAY

**The user got the two
seconds back
they were losing on every
step.**

Nothing exotic. We moved blocking work off the UI thread, cached what we kept reading, and stopped waiting on things that didn't need to be waited on.